MODELAGEM DE PROCESSOS E NEGÓCIOS

AULA 5

Prof.ª Adriana Bastos da Costa

CONVERSA INICIAL

Em nosso estudo, temos discutido sobre os conceitos relacionados com o mapeamento de processos, que ajudam a entender como funcionam as empresas e seus processos.

Anteriormente, reforçamos os conceitos fundamentais de modelagem de processos. Também discutimos sobre diferentes tipos e técnicas utilizados no mapeamento de processos, aprofundando-nos no uso do BPM e da UML. Estudamos, também, o funcionamento de cada técnica, buscando deixar mais clara a diferença entre as duas. Na sequência, vimos com mais detalhes o funcionamento do *Business Process Model and Notation* (BPMN), entendendo como a notação está organizada e estudando um exemplo de modelagem de processo utilizando essa ferramenta. Depois, falamos sobre a UML e seus diversos diagramas, discutindo a importância de cada um para o entendimento de diferentes aspectos do software.

Ou seja, até o momento reforçamos a importância de mapear os processos de negócio antes de entender, planejar tecnicamente e construir um software. Vimos que o uso de uma ferramenta não é obrigatório, mas facilita muito todo o processo de modelagem de negócios.

Nesta abordagem, vamos conversar sobre a importância de documentar em detalhes os requisitos de um software, principalmente a necessidade de descrever claramente as regras de negócios e as restrições existentes nessas regras para que o software funcione corretamente para atender aos usuários.

O conteúdo está dividido em cinco tópicos, que estão relacionados entre si e ajudarão a entender os conceitos propostos.

Fazendo um resumo, trataremos dos seguintes tópicos:

- **Tópico 1** – Introdução e importância de restrições e padrões de negócio na engenharia de software.
- **Tópico 2** – Identificação e documentação de restrições.
- **Tópico 3** – Categorias de regras de negócio.
- **Tópico 4** – Modelagem de restrições e tipos de padrão.
- **Tópico 5** – Gabaritos de padrões de negócio.

TEMA 1 – INTRODUÇÃO E IMPORTÂNCIA DE RESTRIÇÕES E PADRÕES DE NEGÓCIO NA ENGENHARIA DE SOFTWARE

Na engenharia de software, a implementação de restrições e padrões de negócio é crucial para garantir que o desenvolvimento de sistemas de software atenda aos objetivos empresariais, regulatórios e operacionais. Ou seja, é preciso levantar e documentar os requisitos de um software, mas tão importante quanto entender os requisitos é identificar e documentar as restrições que afetam os requisitos.

Restrições de negócio são limitações ou condições específicas que devem ser consideradas e cumpridas durante o desenvolvimento de um software. Elas podem vir de diversas fontes, como requisitos de negócios, requisitos técnicos, regulamentações legais ou condições operacionais. As restrições geralmente não são negociáveis e devem ser estritamente seguidas para garantir que o software atenda às expectativas e aos requisitos estipulados. Alguns exemplos de restrições:

- O sistema deve ser implementado usando a linguagem Java.
- O tempo de resposta para qualquer consulta do usuário não deve exceder 2 segundos.
- O software deve estar em conformidade com a legislação GDPR para proteção de dados.
- O orçamento total do projeto não pode exceder $500.000.

Um outro conceito que também é preciso compreender é o de premissas. Premissas em software são suposições feitas durante o processo de desenvolvimento que são consideradas verdadeiras para fins de planejamento e execução do projeto. Elas servem como base para tomar decisões e estabelecer expectativas. As premissas são críticas para o planejamento de projetos, pois ajudam a definir o contexto no qual o software será desenvolvido e operado.

Já os padrões de negócio são diretrizes ou normas que orientam a forma como o software deve ser desenvolvido e mantido, ou seja, podem ser entendidos com conjuntos de diretrizes, regras e melhores práticas que orientam os desenvolvedores na criação de software de forma consistente, eficiente e de alta qualidade. Esses padrões cobrem diversas áreas do ciclo de vida do software, incluindo codificação, design, testes, documentação e manutenção.

Tanto as restrições e as premissas quanto os padrões de desenvolvimento precisam ser identificados e analisados, para que os impactos reflitam nos requisitos do software.

Vamos discutir e entender quais são os tipos de restrições e os padrões de negócios mais conhecidos. Quando falamos em restrições de negócios, podemos listar como principais:

- **Regulatórias**: requisitos impostos por leis e regulamentos, por exemplo, a LGPD, para proteção de dados.
- **Financeiras**: orçamentos e limites de custo que o projeto não deve exceder.
- **Tecnológicas**: limitações relacionadas às tecnologias que devem ou não ser usadas.
- **Operacionais**: condições operacionais específicas, como tempo de resposta e disponibilidade do sistema.

  Em relação aos padrões de negócios, alguns dos principais são:
- **Padrões de qualidade**: diretrizes que asseguram a qualidade do software, por exemplo, ISO/IEC 25010 para qualidade de produto de software, CMMI ou MPS.Br, como modelos de melhores práticas para construção de software. Ou seja, usar padrões de qualidade em software é fundamental para garantir que os sistemas sejam robustos, seguros, eficientes e atendam às necessidades dos usuários e às expectativas dos clientes.
- **Padrões de desenvolvimento**: normas para codificação, documentação e práticas de desenvolvimento, como o uso de metodologias ágeis, padrões de codificação predefinidos, dependendo da linguagem a ser utilizada, padrões de arquitetura, entre outros. As boas práticas na construção do software garantem mais qualidade e facilita manutenções futuras que deverão ser feitas no código. Elas asseguram a consistência, pois todo o código dentro de um projeto ou até mesmo entre projetos passa a ter um estilo e uma estrutura uniformes, melhorando a qualidade do código, reduzindo a ocorrência de bugs e problemas de manutenção.
- **Padrões de segurança**: requisitos para garantir a segurança do software, como a ISO/IEC 27001, que é um padrão internacional para a gestão da segurança da informação. Define requisitos para estabelecer, implementar, manter e melhorar continuamente um sistema de gestão de

segurança da informação (SGSI). Ou seja, a adoção de padrões de segurança é crucial para proteger sistemas e dados contra ameaças cada vez mais sofisticadas. Eles fornecem uma base estruturada e amplamente reconhecida para a implementação de práticas de segurança robustas e eficazes. Empresas que seguem esses padrões podem não apenas proteger melhor suas informações, mas também ganhar a confiança de clientes e parceiros, e assegurar a conformidade com regulamentações e requisitos legais.

Para entender melhor a importância de identificar as restrições e definir os padrões de negócios, é preciso compreender que seguir restrições regulatórias evita multas e penalidades legais e, no contexto das normas industriais, a aderência a padrões industriais garante que o software atenda a expectativas do mercado e requisitos de certificação.

A qualidade e a facilidade de manutenção também são benefícios que podem ser identificados, pois padrões de desenvolvimento garantem que o código seja consistente e legível, facilitando a manutenção e a evolução do software. Além disso, as restrições e os padrões de qualidade asseguram que o software atende aos critérios necessários para desempenho, segurança, usabilidade e outras características importantes.

O uso de padrões de desenvolvimento e ferramentas consistentes aumenta a produtividade da equipe, reduzindo o tempo necessário para resolver problemas e integrar novos desenvolvedores, pois padrões claros e bem-definidos reduzem a probabilidade de erros e defeitos no software.

Em relação à segurança e confiabilidade, podemos reforçar que os padrões de segurança minimizam vulnerabilidades, protegendo dados sensíveis e garantindo a integridade do sistema. A aderência a padrões de negócio e qualidade aumenta a confiabilidade do software, essencial para sistemas críticos.

Logo, as restrições de negócio garantem que o software atenda aos requisitos específicos do cliente, aumentando a satisfação. Já os padrões de design e usabilidade asseguram que o software seja intuitivo e fácil de usar, mas é preciso que a equipe de desenvolvimento conheça e entenda sobre a importância de seguir restrições e padrões. Além disso, deve-se utilizar ferramentas de análise de código, integração contínua e gestão de conformidade para garantir que os padrões sejam seguidos consistentemente. A realização de

revisões de código e auditorias regulares ajudam a verificar a conformidade com as restrições e padrões estabelecidos, mantendo a qualidade do código construído. Outra boa prática que deve ser seguida é manter a documentação atualizada sobre todos os padrões e restrições e comunicar quaisquer alterações à equipe de desenvolvimento.

Restrições e padrões de negócio são fundamentais na engenharia de software para garantir que o desenvolvimento deste esteja alinhado com os objetivos empresariais, regulatórios e operacionais. Eles ajudam a garantir conformidade, qualidade, eficiência, segurança e satisfação do cliente, tornando-se componentes essenciais para o sucesso de projetos de software. A implementação eficaz dessas diretrizes requer uma abordagem estruturada que inclua definição clara de requisitos, treinamento, uso de ferramentas, revisões regulares e comunicação contínua.

TEMA 2 – IDENTIFICAÇÃO E DOCUMENTAÇÃO DE RESTRIÇÕES

Agora que já entendemos o que são as restrições e como elas impactam o projeto de software, vamos entender como elas devem ser identificadas e documentadas.

Logo, identificar e documentar restrições de um software é uma etapa crucial no processo de desenvolvimento, pois essas restrições influenciam diretamente o design, a implementação e a operação do software. Por ser uma atividade complexa e que precisa seguir um conjunto de passos, é importante definir um processo, que pode ser definido para cada situação de projeto, mas que pode conter os seguintes passos:

- **Passo 1 – Reunir requisitos e entender o contexto**
  - **Reuniões com os envolvidos**: realize reuniões com todos os envolvidos no projeto, sejam eles clientes, sejam usuários finais, gerentes de projeto e equipe de TI, para entender os requisitos do projeto e as limitações que podem impactar o desenvolvimento do software.
  - **Análise de documentação**: analise toda a documentação disponível, como contratos, propostas de projeto, e regulamentos legais, para identificar restrições explícitas.

- **Estudos de viabilidade**: conduza estudos de viabilidade técnica, econômica e operacional para identificar quaisquer limitações no projeto. Mas o que é um estudo de viabilidade, afinal? Bom, um estudo de viabilidade técnica pode ser entendido como sendo uma análise detalhada realizada para determinar se um projeto de software ou sistema pode ser implementado do ponto de vista técnico. Esse estudo examina os aspectos técnicos do projeto, avaliando a disponibilidade e a adequação das tecnologias, recursos, e competências necessárias para a realização do projeto. O objetivo é identificar os potenciais desafios técnicos e garantir que o projeto seja viável e sustentável antes de avançar para fases subsequentes do desenvolvimento.

- **Passo 2 – Classificar restrições** – classifique as restrições em categorias, para organizar e entender melhor seu impacto no projeto. Essa é uma prática essencial no gerenciamento de projetos de software, pois melhora a clareza, facilita a análise e comunicação, e ajuda na identificação e mitigação de riscos, além de contribuir para um planejamento mais eficaz e uma tomada de decisão baseada em informações, resultando em um projeto mais organizado e com maior chance de sucesso. Algumas das principais categorias relacionadas com as restrições em um projeto de software são:
  - **Tecnológicas**:
    - **Compatibilidade**: restrições quanto à compatibilidade com sistemas operacionais, navegadores ou versões de hardware específicas.
    - **Linguagens e Frameworks**: limitações sobre quais linguagens de programação ou frameworks podem ser usados.
    - **Desempenho**: requisitos mínimos de desempenho, como tempos de resposta e uso de recursos.
  - **Legais e regulatórias**:
    - **Conformidade**: necessidade de aderir a regulamentos e padrões como a Lei Geral de Proteção de Dados (LGPD), que se aplica a todas as empresas que operam no território nacional. Já os padrões HIPAA (H*ealth Insurance Portability and Accountability Act*) e PCI DSS (*Payment Card Industry Data Security Standard*) são exemplos de padrões de

regulamentações e diretrizes que visam proteger informações sensíveis em setores específicos. São críticos para garantir a segurança e a privacidade dos dados, e seu cumprimento é obrigatório para as organizações que operam nos respectivos setores.

- **Licenciamento**: o licenciamento refere-se ao processo pelo qual o proprietário de um produto, serviço, tecnologia ou propriedade intelectual concede permissão a outra parte para usar esse recurso sob determinadas condições e termos. Esse processo é comum em diversos setores, incluindo software, música, patentes, marcas comerciais e outros tipos de propriedade intelectual. No contexto de software, os licenciamentos são particularmente importantes e vêm em várias formas, cada uma com suas próprias regras e condições. Ou seja, no contexto de software, o licenciamento está relacionado com as restrições impostas por licenças de software.

- **Financeiras**: limitações de custo que afetam escolhas tecnológicas e o escopo do projeto. O orçamento para concluir o projeto é de até R$ 150.000,00.
- **Prazo**: restrições de tempo que impactam a entrega de funcionalidades; portanto, o final do projeto. Por exemplo, o projeto precisa finalizar em até 1 mês antes do Natal.
- **Organizacionais**:
  - **Políticas internas**: diretrizes e políticas organizacionais que influenciam o desenvolvimento. Por exemplo, uma empresa de serviços financeiros está desenvolvendo um novo sistema de gerenciamento de clientes para melhor atender às necessidades da área de marketing e aumentar a eficiência operacional do atendimento aos clientes. A política interna de segurança da informação da empresa é uma das restrições que devem ser consideradas ao planejar e implementar o projeto.
  - **Recursos Humanos**: disponibilidade e competência da equipe de desenvolvimento. Por exemplo, em um projeto onde os requisitos de segurança são muito críticos, a limitação de

especialistas em segurança pode atrasar fases críticas do projeto, como a implementação de medidas de segurança, revisões de código de segurança e testes de penetração. Isso pode resultar em um cronograma estendido para garantir que todas as questões de segurança sejam adequadamente tratadas.

- **Operacionais**:
  - **Disponibilidade**: Requisitos de alta disponibilidade e recuperação de desastres.
  - **Escalabilidade**: limitações na capacidade de escalar o software.

- **Passo 3 – Documentar restrições** – documente as restrições de forma clara e detalhada, de forma a ajudar todas as partes interessadas a entenderem os limites do projeto. Não existe uma forma única para documentar as restrições, mas, para facilitar o entendimento, vamos explorar um formato com algumas informações sugeridas, que ajudam a documentar de forma clara as restrições identificadas:
  - **Identificação da restrição**: dê um identificador único para cada restrição (por exemplo, R1, R2 etc.).
  - **Descrição da restrição**: forneça uma descrição detalhada da restrição. Explique o que ela implica e por que é importante.
  - **Categoria da restrição**: classifique a restrição (tecnológica, legal, financeira, organizacional, operacional).
  - **Impacto**: descreva o impacto da restrição no projeto. Como ela afeta o design, desenvolvimento e operação do software? O impacto pode ser financeiro ou não.
  - **Stakeholders relevantes**: identifique os stakeholders que estão interessados ou que impuseram a restrição. Lembrando que Stakeholders ou partes interessadas, em português, são todas as pessoas, grupos ou entidades que têm interesse ou são afetados pelas atividades e resultados do projeto. Eles podem influenciar ou ser influenciados pelo sucesso ou fracasso de uma iniciativa. Stakeholders podem incluir tanto indivíduos internos à organização quanto externos a ela.

- **Data de identificação**: registre a data em que a restrição foi identificada.
- **Responsável**: atribua um responsável pela gestão ou mitigação da restrição.
- **Notas adicionais**: inclua qualquer informação adicional relevante para a compreensão e gestão da restrição.

- **Passo 4 – Revisão e validação**
  - **Revisão interna**: revise as restrições com a equipe de desenvolvimento e outros stakeholders internos para garantir que todas as restrições foram corretamente identificadas e documentadas.
  - **Validação com stakeholders**: apresente a documentação de restrições aos stakeholders para validação e confirmação de que todas as restrições importantes foram capturadas e corretamente documentadas.
- **Passo 5 – Gestão contínua das restrições**
  - **Monitoramento**: monitore as restrições ao longo do ciclo de vida do projeto para garantir que continuam sendo respeitadas.
  - **Atualizações**: atualize a documentação de restrições conforme novas restrições sejam identificadas ou mudanças ocorram nas restrições existentes. Nesse caso, é preciso revisar também o planejamento do projeto, para garantir que as alterações das restrições ou as novas restrições não gerem maiores impactos no projeto. Além disso, uma alteração ou a inclusão de uma nova restrição pode impactar significativamente a solução técnica definida para o projeto.
  - **Comunicação**: mantenha todas as partes interessadas informadas sobre quaisquer alterações nas restrições.

Anteriormente, estudamos sobre a linguagem de modelagem UML, que modela vários aspectos do software. Para identificar e especificar as restrições de um software podemos usar o OCL – *Object Constraint Language*. O OCL é uma linguagem de especificação utilizada para definir restrições em modelos UML.

Os métodos para identificar restrições utilizando OCL envolvem a definição de regras e restrições que os objetos em um sistema devem seguir. Isso ajuda a garantir a consistência e a integridade do sistema. OCL funciona permitindo que os analistas especifiquem restrições textuais que podem ser

verificadas automaticamente em modelos durante o processo de desenvolvimento de software. Vamos estudar alguns exemplos de uso do OCL para documentar restrições:

Exemplo 1:

“O saldo de uma conta bancária não pode ser negativo.”

Contexto ContaBancaria
Inv SaldoNaoNegativo: self.saldo >= 0

Neste exemplo, "context ContaBancaria" define que a restrição se aplica à classe ContaBancaria e "inv SaldoNaoNegativo" é o nome da restrição. A expressão "self.saldo >= 0" especifica que o saldo da conta bancária (representado por "self.saldo") deve ser maior ou igual a zero. Essa restrição pode ser verificada automaticamente no modelo para garantir que o saldo nunca seja negativo.

Exemplo 2:

"Para um pedido ser considerado válido, a data de entrega deve ser posterior à data atual."

Contexto pedido
inv DataEntregaPosteriorAtual: self.dataEntrega > OclAny.currentDate()

Neste exemplo, a restrição "DataEntregaPosteriorAtual" é aplicada à classe Pedido. A expressão "self.dataEntrega > OclAny.currentDate()" garante que a data de entrega do pedido seja posterior à data atual, verificando automaticamente essa condição durante o desenvolvimento do software.

Identificar e documentar restrições de software é uma prática essencial para garantir que o desenvolvimento ocorra dentro dos limites estabelecidos e

atenda às expectativas dos stakeholders.

Um processo estruturado de identificação e documentação ajuda a mitigar riscos, planejar de forma eficaz e assegurar que todos os envolvidos tenham uma compreensão clara das limitações e condições que impactam o projeto.

TEMA 3 – CATEGORIAS DE REGRAS DE NEGÓCIO

Quando estudamos sobre requisitos de software, é importante entender o que são regras de negócio. Dessa forma, podemos dizer que regras de negócios são declarações específicas que definem ou restringem aspectos de uma empresa, influenciando comportamentos e decisões dentro da organização. Em projetos de software, essas regras orientam como o software deve operar para atender às necessidades da organização, garantindo que ele suporte os processos de negócios de maneira eficaz e conforme as políticas da empresa.

Para que as regras de negócios sejam completamente identificadas e compreendidas, é preciso respeitar algumas características:

- **Clareza e especificidade**: regras de negócios devem ser claramente definidas e específicas para evitar ambiguidades nos requisitos do software.
- **Independência de implementação**: descrevem *o que* deve ser feito, mas não necessariamente *como* deve ser feito. A implementação específica é determinada pela equipe de desenvolvimento, para garantir a melhor solução técnica.
- **Mensurabilidade e testabilidade**: devem ser mensuráveis e testáveis para garantir que podem ser verificadas durante o desenvolvimento, quando o software estiver construído e os testes forem iniciados, e mesmo após a implantação do software em ambiente de produção.
- **Derivadas das políticas e objetivos de negócio**: regras de negócios podem ser derivadas das políticas, metas e regulamentos que a organização deve seguir.

A importância das regras de negócios é muito grande e a sua correta e completa identificação agrega valor no alinhamento com os objetivos de negócio, pois asseguram que o software suporte e reflita os objetivos e políticas da empresa. Elas também garantem que todos os processos de negócios sejam executados de maneira consistente e em conformidade com regulamentos

internos e externos, definindo claramente as regras e, por consequência, reduzindo a ambiguidade e minimizando a ocorrência de erros. Além disso, regras bem documentadas facilitam a atualização do software à medida que as políticas de negócios mudam.

As regras de negócios podem ser classificadas em várias categorias, cada uma focada em um aspecto específico dos processos de negócios. Essas categorias ajudam a organizar e estruturar as regras de negócios para que possam ser gerenciadas e implementadas de maneira eficaz em sistemas de software. As principais categorias incluem:

- **Regras de definição**: definem termos e conceitos usados nos processos de negócios. Exemplo: "Um cliente ativo é definido como um cliente que realizou pelo menos uma compra nos últimos 12 meses".
- **Regras de restrição**: impõem limitações sobre o que pode ou não ser feito. Exemplo: "Os pedidos que excedem R$ 10.000,00 devem ser aprovados por um gerente".
- **Regras de ação/procedimento**: especificam ações ou procedimentos que devem ser seguidos em determinadas condições. Exemplos: "Se um pagamento não for recebido dentro de 30 dias, enviar um aviso de pagamento atrasado ao cliente"; "Enviar um e-mail de agradecimento automático para cada cliente que fizer uma compra superior a R$ 500,00".
- **Regras de inferência**: derivam novos dados ou conclusões a partir de dados existentes. Exemplo: "Se um cliente faz mais de três reclamações em um mês, classifique-o como um cliente de alto risco".
- **Regras de cálculo**: definem como certos cálculos devem ser realizados. Exemplo: "Calcular o desconto anual do cliente com base no total de compras realizadas no ano corrente".
- **Regras de validação**: garantem que os dados inseridos no sistema atendem a certos critérios ou padrões. Exemplo: "O campo 'data de nascimento' deve conter uma data válida e não pode estar no futuro".
- **Regras de autorização**: definem quem tem permissão para realizar certas ações ou acessar certos dados. Exemplo: "Somente gerentes podem aprovar descontos superiores a 20%".

Para deixar mais claro o entendimento sobre o que estudamos até agora em relação às regras de negócios, vamos analisar alguns exemplos de software

e suas regras de negócio, lembrando que as regras de negócios variam de empresa para empresa, pois elas refletem a forma com a empresa estrutura seus processos e suas políticas.

- Exemplo 1: Sistema de gestão de vendas

➢ **Regra de restrição**: "Um representante de vendas não pode conceder um desconto superior a 20% sem aprovação do gerente".

➢ **Regra de definição**: "Um 'cliente VIP' é um cliente que gasta mais de R$ 50.000,00, anualmente".

➢ **Regra de ação**: "Enviar um e-mail de agradecimento automático para cada cliente que fizer uma compra superior a R$ 500".

- Exemplo 2: Sistema bancário

➢ **Regra de Restrição**: "Transações superiores a R$10,000 devem ser reportadas ao departamento de compliance".

➢ **Regra de definição**: "Uma 'conta inativa' é uma conta que não teve transações nos últimos 24 meses".

➢ **Regra de cálculo**: "Calcular os juros mensais das contas de poupança com base na taxa de juros anual dividida por 12".

A identificação das regras de negócio ocorre na fase de análise de requisitos, no ciclo de vida de desenvolvimento do software, mas pode também fazer parte de outras fases ao longo da construção do projeto. Ou seja, as regras de negócio, assim como os requisitos, precisam ser gerenciadas ao longo de todo o processo de desenvolvimento de um projeto de software. Vamos entender como as regras de negócio se comportam ao longo do ciclo de vida de um software.

- **Análise de requisitos**: durante a fase de coleta de requisitos, identificar e documentar todas as regras de negócios relevantes.
- **Modelagem de processos de negócios**: usar diagramas de fluxo de trabalho e modelos de processos para visualizar onde e como as regras de negócios se aplicam.
- **Implementação**: codificar as regras de negócios no software, garantindo que elas sejam testáveis.

- **Testes e validação**: criar casos de teste específicos para validar que as regras de negócios foram implementadas corretamente e funcionam conforme esperado.
- **Documentação**: manter uma documentação clara e acessível das regras de negócios para facilitar a manutenção futura e assegurar que todos os membros da equipe entendam as restrições e expectativas.

Categorizar as regras de negócios é importante por vários motivos, entre eles:

- **Organização e clareza**: facilita a organização das regras, tornando-as mais claras e compreensíveis.
- **Implementação eficiente**: ajuda os desenvolvedores a entenderem onde e como cada regra deve ser implementada no sistema.
- **Manutenção e atualização**: simplifica a manutenção e a atualização das regras, permitindo identificar rapidamente quais regras podem ser afetadas por mudanças nas políticas de negócios.
- **Consistência e conformidade**: assegura que todas as áreas do negócio estejam aderindo consistentemente às mesmas regras, ajudando na conformidade com regulamentações.

Logo, as regras de negócios são componentes cruciais em projetos de software, assegurando que o sistema desenvolvido esteja coerente com as políticas e objetivos da organização. Elas definem claramente como os processos devem funcionar, estabelecendo um conjunto de expectativas claras e mensuráveis. A gestão eficaz dessas regras desde o início do projeto até a implementação e manutenção contínuas é essencial para o sucesso de qualquer sistema de software.

Percebemos, então, que as regras de negócios são partes inerentes dos requisitos de um software, e são essenciais para garantir que os processos de negócios sejam executados de maneira correta, eficiente e em conformidade com as políticas da organização. Categorizar essas regras ajuda a organizar e gerenciar melhor a implementação no software, assegurando que cada aspecto do negócio seja adequadamente suportado pela solução tecnológica definida.

TEMA 4 – MODELAGEM DE RESTRIÇÕES E TIPOS DE PADRÕES

A modelagem de restrições e tipos de padrões no contexto de desenvolvimento de software refere-se ao processo de definir e aplicar restrições e padrões de design para garantir a consistência, a qualidade e a eficiência do software.

Isso envolve identificar as restrições que o sistema deve seguir e os padrões de design que serão aplicados para resolver problemas comuns de forma eficaz.

4.1. Modelagem de restrições

A modelagem de restrições ajuda a garantir que o software atenda aos requisitos específicos do projeto, enquanto a aplicação de padrões de design ajuda a promover a consistência, reutilização e manutenibilidade do código.

Em tópicos anteriores, vimos que modelar restrições pode ser bastante complexo se levarmos em conta que é preciso identificar todas as restrições e modelá-las para que sejam corretamente codificadas no software.

Dessa forma, para modelar restrições no desenvolvimento de software, é preciso seguir alguns passos, para organizar as atividades e garantir que as restrições sejam identificadas, entendidas, documentadas e traduzidas em linhas de código que vão fazer o software gerar o resultado correto. Vamos analisar alguns dos principais passos que podem ser seguidos para modelar as restrições em um software:

- **Identificar as restrições**: comece identificando as restrições e requisitos específicos do sistema. Isso pode incluir restrições de desempenho, segurança, integridade dos dados, entre outras.
- **Documentar as restrições**: registre todas as restrições de forma clara e precisa. Utilize diagramas, documentos ou ferramentas de modelagem para documentar as restrições de forma compreensível.
- **Validar as restrições**: verifique se as restrições são consistentes, pertinentes e alcançáveis. Certifique-se de que todas as partes interessadas concordem com as restrições identificadas.
- **Implementar as restrições**: durante a fase de design e implementação do software, garanta que as restrições sejam levadas em consideração.

Isso pode exigir a aplicação de restrições de integridade, validação de entrada de dados, entre outros mecanismos.

- **Testar e validar as restrições**: após a implementação, teste e valide as restrições para garantir que o software esteja em conformidade com as restrições definidas.

Ao seguir esses passos e manter um controle claro das restrições ao longo do ciclo de desenvolvimento de software, você poderá modelar de forma eficaz as restrições necessárias para o seu sistema.

4.2. Padrões de projeto

Quando desenvolvemos um software, é natural encontrarmos desafios, pois construir software é uma atividade bastante complexa. Conforme vamos adquirindo experiência, percebemos que determinados problemas são recorrentes, e se repetem mesmo em projetos diferentes. Dessa forma, podemos utilizar esse conhecimento para resolver o problema atual. É como se houvesse um padrão para solucionar certos tipos de problemas encontrados ao longo do desenvolvimento dos softwares. Esses padrões são também conhecidos como *Design Patterns* ou Padrões de Projetos.

Mas como esses conceitos de padrões de projetos surgiram e evoluíram? Vamos conhecer um pouco dessa história, que não é recente, remonta dos anos 1970, quando os arquitetos Christopher Alexander, Sara Ishikawa e Murray Silverstein, em 1978, escreveram um livro chamado *A Pattern Language*: *Towns, Buildings, Construction*, publicado em português com o nome *Uma Linguagem de Padrões*.

Nesse livro foram catalogados cerca de 253 tipos de problemas ou desafios de projetos de software, e os autores analisaram cada uma dessas situações, descrevendo-as em detalhe e propondo uma solução padrão. É incrível como os problemas de software são antigos e complexos, mas as coisas acontecem e vão evoluindo aos poucos.

Durante uma das edições da conferência anual da área de computação, chamada de OOPSLA – *Object-Oriented Programming*, Systems, *Languages, and Applications*, ocorrida em 1987, o engenheiro de software Kent Beck, com Ward Cunningham apresentaram uma palestra intitulada "*Using Pattern Languages for Object-Oriented Programs*", que em uma tradução livre para o

português pode ser chamada de “Utilizando a linguagem dos padrões para programação orientada a objetos”. Nessa palestra, eles organizaram os conceitos iniciados em 1978 e propuseram cinco padrões de projetos no campo da ciência da computação.

Kent Beck sempre foi um estudioso e visionário, pois foi ele que, posteriormente, foi um dos criadores das metodologias XP – *Extreme Programming* e TDD – *Test Driven Development*. Ambas amplamente utilizadas em projetos que seguem as metodologias ágeis de desenvolvimento de software.

Mas os conceitos de Kent Beck e Ward Cunningham só ficaram realmente conhecidos em 1994, quando os engenheiros de software Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides escreveram o livro *Design Patterns: Elements of Reusable Object-Oriented Software* com o objetivo de catalogar problemas comuns aos projetos de desenvolvimento de software e as formas de resolver esses problemas.

Nesse novo livro, seus autores definiram e catalogaram 23 padrões que utilizaram ao longo de suas carreiras. Esse livro é um sucesso até os dias de hoje, teve muitos exemplares vendidos e já foi publicado em mais de 12 idiomas. No Brasil foi publicado com o nome *Padrões de Projeto – Soluções Reutilizáveis de Software Orientado a Objetos*. Os autores do livro ficaram conhecidos como Gang of Four (Gangue dos quatro) ou “GoF”. Depois disso, muitos outros livros surgiram, alguns criticando alguns desses padrões, e outros divulgando novos padrões. Ou seja, os conceitos continuam em evolução, como vários outros conceitos relacionados com o desenvolvimento de software.

Desde então, *Design Patterns* tem sido um tema bastante estudado por programadores e arquitetos de software pelo mundo todo.

Mas voltando ao nosso assunto principal, é possível compreender que cada tipo de padrão em desenvolvimento de software tem um objetivo de fornecer soluções comprovadas para problemas recorrentes e promover boas práticas de design e desenvolvimento de sistemas de software. Vamos compreender o objetivo de cada um dos principais tipos de padrão utilizados atualmente no desenvolvimento de um software:

- Padrões de criação: fornecem vários mecanismos de criação de objetos, que aumentam a flexibilidade e a reutilização de código já existente. Fornece uma interface para criar objetos em uma superclasse, mas

permite que as subclasses alterem o tipo de objetos que serão criados. Alguns padrões criacionais mais utilizados são:

  - *Singleton*: garante a existência de apenas uma instância de uma classe e fornece um ponto global de acesso à essa instância.
  - *Factory Method*: define uma interface para criar objetos, mas permite que subclasses decidam qual classe instanciar.
  - *Abstract Factory*: fornece uma interface para criar famílias de objetos relacionados sem especificar suas classes concretas.
  - *Builder*: separa a construção de um objeto complexo de sua representação, permitindo a criação de diferentes tipos e representações do objeto.
  - *Prototype*: permite a criação de novos objetos a partir de cópias de objetos existentes.

- **Padrões de estrutura**: os padrões dessa categoria se preocupam em melhor organizar a estrutura das classes e os relacionamentos entre classes e objetos. Alguns padrões estruturais mais utilizados são:
  - *Adapter*: permite que objetos com interfaces incompatíveis trabalhem juntos.
  - *Bridge*: separa uma abstração de sua implementação, permitindo que ambas variem independentemente.
  - *Composite*: permite tratar objetos individuais e composições de objetos de maneira uniforme.
  - *Decorator*: adiciona responsabilidades a objetos de forma dinâmica.
  - *Facade*: fornece uma interface simplificada para um sistema complexo.

- **Padrões de comportamento**: os padrões dessa categoria atuam diretamente na delegação de responsabilidades, definindo como os objetos devem se comportar e se comunicar. Alguns padrões comportamentais mais utilizados são:
  - *Observer*: define uma dependência um-para-muitos entre objetos para que, quando um objeto mude de estado, todos os seus dependentes sejam notificados e atualizados automaticamente.
  - *Strategy*: permite que um algoritmo seja selecionado durante a execução de um programa.

- *Template Method*: define o esqueleto de um algoritmo em uma operação, delegando a implementação de passos individuais para subclasses.
- *Chain of Responsibility*: permite que vários objetos processem uma solicitação sem especificar explicitamente o receptor.

- **Padrões arquiteturais**: são peças fundamentais para concretizar toda a abstração, e a escolha adequada ao modelo de negócio é essencial, pois evita retrabalho. É possível até trabalhar com padrões de arquitetura diferentes no mesmo software, seja para atender determinados requisitos do negócio, facilitar a manutenção ou aproveitar vantagens de diferentes tecnologias, diferentes padrões podem ser selecionados e utilizados. De forma geral, um padrão arquitetural é uma solução já estabelecida para desenvolvimento de softwares, sendo um modelo reutilizável para problemas já conhecidos. Esses padrões já foram estudados, testados em projetos reais e passaram por melhorias, ou seja, são soluções difundidas e aceitas no mercado. Alguns dos principais padrões arquiteturais utilizados atualmente são:
  - *Model-View-Controller* (MVC): separa a aplicação em três componentes distintos: Modelo, Visão e Controlador.
  - *Model-View-ViewModel* (MVVM): similar ao MVC, mas com a adição de um Modelo de Visão que atua como um adaptador entre a Visão e o Modelo.
  - *Microservices*: arquitetura que organiza uma aplicação como um conjunto de serviços pequenos e independentes.
  - *Event-Driven Architecture*: baseada na produção, detecção, consumo e reação a eventos no software, permitindo a comunicação assíncrona entre componentes.

Esses padrões são ferramentas valiosas no desenvolvimento de software, fornecendo soluções comprovadas para desafios comuns e promovendo a escrita de código mais coeso, flexível e fácil de manter. É uma boa forma de ganhar produtividade, por meio da reutilização de soluções preexistentes para problemas conhecidos.

TEMA 5 – GABARITOS DE PADRÕES DE NEGÓCIO

Gabaritos de padrões de negócio ou apenas padrões de negócios são modelos ou exemplos predefinidos que representam as melhores práticas e soluções para problemas comuns em um determinado domínio de negócios. Eles são utilizados para orientar e agilizar o desenvolvimento de software, fornecendo diretrizes claras e estruturadas para a implementação de funcionalidades específicas. Esses gabaritos podem incluir modelos de processos, regras de negócio, fluxos de trabalho, entre outros elementos que ajudam a padronizar e otimizar a maneira como as atividades de negócio são realizadas.

Os padrões de negócio são frequentemente utilizados no contexto do desenvolvimento de software para garantir a consistência, eficiência e qualidade das soluções de software. Mas no contexto de desenvolvimento de software, também podemos chamar os padrões de negócio de *Design Patterns* ou padrões de projetos.

Eles ajudam a incorporar as melhores práticas de negócio diretamente no desenvolvimento de software, permitindo que as empresas implementem processos e funcionalidades de forma mais eficaz e alinhada com suas necessidades e objetivos comerciais. Assim, os padrões de negócio desempenham um papel importante na criação de software que atenda às exigências e expectativas do negócio.

Resumindo, os padrões de projetos são soluções generalistas para problemas recorrentes durante o desenvolvimento de um software. Não se trata de um framework ou um código pronto, mas de uma definição de alto nível de como um problema comum pode ser tecnicamente solucionado.

No tópico anterior estudamos que existem padrões de projeto e padrões arquiteturais, ambos aplicados no desenvolvimento de software. Vamos agora explorar, com mais detalhe, alguns exemplos de padrões no contexto de software:

- **Padrão de Projeto *Factory Method***: é um padrão criacional. Define uma interface para criar um objeto, mas permite que as subclasses escolham o tipo de objeto a ser criado. O funcionamento ocorre da seguinte forma: uma classe abstrata define um método de fábrica que as subclasses devem implementar, cada subclasse pode fornecer uma implementação

específica para criar um objeto, o código-cliente chama o método de fábrica na classe abstrata para instanciar o objeto desejado, sem precisar conhecer a classe concreta.

- **Padrão de Projeto *Singleton***: é um padrão criacional. Garante que uma classe tenha apenas uma instância e fornece um ponto global de acesso a essa instância. O funcionamento ocorre da seguinte forma: o padrão *Singleton* é implementado garantindo que a classe tenha um construtor privado para evitar instâncias extras e fornecendo um método estático que retorna a única instância válida da classe. Quando o padrão *Singleton* é utilizado, sempre que um objeto *Singleton* é necessário, o método estático é invocado para garantir que apenas uma instância da classe seja criada e usada durante toda a execução do programa.
- **Padrão de Projeto *Adapter***: o *Adapter* é um padrão de projeto estrutural, que permite a colaboração de objetos incompatíveis originalmente. O *Adapter* atua como um empacotador entre dois objetos, permitindo que eles consigam se comunicar. Ele captura chamadas para um objeto e as deixa reconhecíveis tanto em formato como em interface para o segundo objeto. De maneira resumida, ele realmente funciona como um adaptador que permite a comunicação entre objetos diferentes.
- **Padrão de Projeto *Observer*:** é um padrão comportamental que permite que qualquer objeto que implemente a interface do assinante possa se inscrever para notificações de eventos em objetos da publicadora. É possível adicionar o mecanismo de inscrição em seus botões, permitindo que o cliente coloque seu próprio código por meio de classes assinantes customizadas. Ou seja, de forma resumida, podemos dizer que o *observer* é um padrão que permite que um objeto notifique outros objetos sobre alterações em seu estado. Ele fornece uma maneira de assinar e cancelar a assinatura desses eventos para qualquer objeto que implemente uma interface de assinante. Ou seja, ele é útil em situações em que vários objetos precisam ser notificados sobre mudanças em um objeto central. Por exemplo, em um sistema de estoque, vários objetos, como a interface do usuário, o banco de dados e os relatórios, podem precisar ser atualizados sempre que o estoque de um produto mudar, e isso é possível de maneira padronizada pelo uso do *observer*.

- **Padrão de arquitetura MVC (*Model-View-Controller*)**: um padrão de arquitetura de software que separa a lógica de negócio (*Model*), a interface do usuário *(View*) e a lógica de controle (*Controlle*r) em componentes distintos. O funcionamento ocorre da seguinte forma: usuário interage com a *View*, a *View* envia comandos para o *Controller*, o *Controlle*r atualiza o Modelo com os comandos recebidos e, por último, o Modelo atualizado notifica a *View* para atualizar a interface do usuário.
    - ***Model* (Modelo)**: representa a lógica de negócio e os dados do aplicativo.
    - ***View* (Visão)**: é a interface do usuário que exibe as informações ao usuário.
    - ***Controller* (Controlador)**: recebe entradas do usuário, interage com o modelo e atualiza a visualização.
- **Padrão de arquitetura em camadas**: divide o software em camadas, como camada de apresentação, de lógica de negócio e de acesso a dados, para facilitar a manutenção e a escalabilidade do sistema. O funcionamento ocorre da seguinte forma: cada camada tem uma responsabilidade clara e se comunica com camadas adjacentes de acordo com regras bem-definidas. Por exemplo, a camada de apresentação lida com a interação com o usuário; a camada de lógica de negócio processa as regras de negócio; e a camada de acesso a dados gerencia a interação com o banco de dados. Esse padrão é um tipo de arquitetura MVC.
- **Padrão de arquitetura orientada a serviços (SOA)**: o sistema é dividido em serviços independentes, sendo cada serviço responsável por uma funcionalidade específica. Em seguida, há o desenvolvimento de interfaces bem-definidas para que o diálogo entre os serviços ocorra sem precisar depender de uma linguagem de programação ou plataforma específica. A ideia é que um software preste um serviço para outro serviço. Onde um serviço é uma função independente e bem-definida que representa uma unidade de funcionalidade de um negócio.
- **Arquitetura monolítica**: é um único bloco criado de forma vertical com todos os componentes, funcionalidades e estruturas implementadas em conjunto, isso significa que nesse bloco as funcionalidades podem rodar em um único processo, ou se for modularizado, todas as camadas do software são interdependentes. Cada parte do software modificada pode

interferir no comportamento do todo. Uma aplicação web pode ser definida como um monolito de acordo com o seu pacote de publicação para a produção. Um sistema monolítico com um único processo tem toda sua base de código executada dentro de um único processo. As funcionalidades do sistema, devido à sua integração, compartilham recursos e o mesmo espaço de endereçamento de memória em um contexto de único processo em execução.

- **Arquitetura baseada em microsserviços**: os microsserviços podem ser considerados uma variante da arquitetura orientada a serviços e o oposto da arquitetura monolítica, pois encontramos serviços ainda menores e mais independentes, com linguagens e plataformas distintas. Nesse padrão, os serviços foram decompostos em serviços menores, chamados de microsserviços, trazendo mais flexibilidade para o desenvolvimento e permitindo a implementação de novos serviços. A arquitetura em microsserviços permite o uso de diferentes tecnologias e linguagens.

A definição de quais padrões adotar depende da necessidade do projeto. A melhor solução técnica é uma definição de ocorre ao longo da fase de análise e projeto, do ciclo de vida de desenvolvimento de software, e a modelagem dos processos de negócios é a porta de entrada para compreender as necessidades do negócio e dos usuários, traduzindo isso em um software robusto, completo e correto para agregar valor ao cliente.

FINALIZANDO

Esperamos, ao final desta abordagem, que os conceitos discutidos tenham ficado claros. O mais importante é entender o que são restrições, como elas devem ser identificadas, documentadas e inseridas na modelagem do software, de forma a garantir que a construção do software atenderá também às restrições existentes.

Além disso, espero que os conceitos relacionados com os padrões utilizados no desenvolvimento de software tenham sido entendidos e os benefícios do uso desses padrões sejam aproveitados nos projetos que vocês venham a desenvolver ao longo da carreira.

O conhecimento sobre os padrões que podem ser utilizados e o objetivo de cada um apoia a tomada de decisão assertiva em relação à melhor solução técnica possível, aumentando assim a qualidade do software.

É importante ressaltar que a escolha do melhor padrão para ser utilizado no projeto começa pela compreensão dos requisitos que serão transformados em solução tecnológica. Já sabemos que utilizar os modelos e padrões ajudam muito a desenvolver softwares com qualidade, mas para isso é preciso conhecer o seu modelo de negócio, as limitações e as vantagens de cada padrão. É preciso também ter em mente que a arquitetura de software não é um formato fechado, não é engessado, muito pelo contrário, ela permite ter a liberdade para tomar as melhores decisões em seu código. Ela é uma base, um direcionador que vai ajudar a organizar a solução da melhor forma possível.

Em conteúdos posteriores, discutiremos outros assuntos que agregarão conhecimento no entendimento geral sobre os conceitos relacionados com a modelagem de processos de negócio.

REFERÊNCIAS

BECKER, J.; ROSEMANN, M.; VON UTHMANN, C. Guidelines of business process modeling. *In*: VAN DER AALST, W.; DESEL, J.; OBERWEIS, A. (ed.). **Business Process Management**. New York: Springer Berlin Heidelberg, 2000. p. 30-49.

COHN, M. **User Stories Applied for Agile Software Development**. Assison-Wesley Professional, 2004.

GANE, C.; SARSON, T. **Análise Estruturada de Sistemas**. São Paulo: LTC, 1983.

YOURDON, E. **Análise Estruturada Moderna**. Rio de Janeiro: Editora Campus. (Tradução da Terceira Edição Americana)

LAPLANTE, P. A. **Requirements engineering for software and systems**. Boca Raton: CRC Press, 2013.

MALL, R. **Fundamentals of software engineering**. New Delhi: PHI Learning, 2014.

OGUNNAIKE, B. A.; RAY, W. H. **Process dynamics, modeling, and control**. Oxford: Oxford University Press, 1994.

PAIM, R. et al. **Gestão de Processos**: pensar, agir e aprender. Porto Alegre: Bookman, 2009. 328 p.

PIZZA, W. R. **A metodologia Business Process Management (BPM) e sua importância para as organizações**. 2012. Monografia (Curso de Tecnologia em Processamento de Dados) – Faculdade de Tecnologia de São Paulo – Fatec SP, São Paulo, 2012.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software**: Uma Abordagem Profissional. 8. ed. Mc Graw Hill Educattion, 2016.

ROSEMANN, M.; BROCKE, J. V.; HONORATO, B. **Manual de BPM**: gestão de processos de negócio. Porto Alegre: Bookman, 2013.

SILVER, B.; RICHARD, B. **BPMN method and style**. Aptos: Cody-Cassidy Press, 2009. v. 2.

WIEGERS, K.; BEATTY, J. **Software requirements**. 3. ed. London: Pearson Education, 2013.